

Breakout – ECE3300 Final Project

Nguyen Pham, Aaron Diller, Ryan Ikeda, Angelo Duenas

Description of the System

This project emulates the retro game “Breakout”, where one player controls a small paddle with the objective of deflecting the ball until all blocks above have been hit. The challenge is to accomplish this objective without failing to deflect the ball, as doing so will lose the game. The FPGA development board serves as both the game’s processor and player’s controller.

Explanation of All Required I/Os

- Switches

One switch is used to change the state of the 7-segments displays. In one state, the 7-segment displays show the player’s current score in their current round. When the switch is flipped, the 7-segment displays instead shows the accumulated score throughout all rounds the player has played.

- Push Buttons

The push buttons serve as the main method of controlling the motion of the paddle from left to right, as well as resetting the game.

- LEDs

The LEDs are utilized as an indicator for the number of lives that the player has. The player begins with 3 lives, and each time the ball drops into the dead zone, the player loses one life. When the player loses all lives, the game ends in a loss.

- 7-Segment Display

The 7-segment displays are used to indicate the player’s score. With each block broken, the player’s score increases by 1. When the game completes, either through a win or loss, the display is reset to 0.

- UART

The UART serves as a secondary controller for the player’s paddle, allowing the player to move the paddle using the W, A, S, and D keys of a keyboard. We also implemented a “Cheat code” where if you press the keyboard in series “WWSS”, the blocks will reset to its original amount (24)

- VGA

The VGA relays the game’s graphics to a monitor. This allows the player to track the ball’s movements and respond by accurately deflecting the ball back towards the blocks.

Explanation of FSM

The FSM works in conjunction with the LEDs to implement the 3-lives system. When the condition of the ball inside the dead zone is met, the FSM moves from the “three lives” state into the “two lives” state, and sequentially from “two lives” state into “one life” state when the ball enters the dead zone again. Upon reaching this condition a third time, the FSM reaches the “dead” state, in which the player has lost. The game then resets and moves the FSM machine back into the “three lives” state for the game to repeat again.

Explanation of FIFO

FIFO is utilized to read a specific cheat code from the UART. If the specific sequence “up up down down (WWSS)” is typed, then the game enables the cheat code and replaces all of the broken blocks back, resetting the score. The UART receiver accepts keyboard commands, and the FIFO stores the received movement characters before the controller processes them. Every time we hit a specific keyboard (W,A,S,D), it is loaded into FIFO, and then pushed out into the FPGA to control the paddle movement.

Explanation of Modeling Styles Used

Behavioral

The project uses behavioral modeling in the sequential logic that controls game state, movement, scoring, UART handling, and VGA timing. Several modules use always @(posedge clk or posedge rst) blocks to describe register-based behavior that changes on the clock edge or resets asynchronously. For example, ball.v updates the ball’s xloc, yloc, xdir, and ydir based on collision conditions, while paddle.v updates paddle position based on movement commands. The life_fsm.v module is also behavioral because it describes the lives system as a state machine with states for three lives, two lives, one life, and dead. The UART controller and FIFO also use clocked always blocks to store received keyboard commands and process them in order.

Structural

The project uses structural modeling in the top-level design by connecting smaller modules together to build the full Breakout system. The top_vga.v module instantiates the VGA timing generator, UART WASD controller, paddle, seven-segment score display, blocks, ball, and life FSM. These modules communicate through wires such as hcount, vcount, draw_ball, draw_block, broken, move, respawn_ball, and reset_blocks. This structural style makes the system modular because each major hardware component performs one role, and the top module connects them into the complete game. Structural modeling is also used in doubdab_8bits.v, which creates several dd_add3 submodules and wires them together to implement the double-dabble binary-to-BCD conversion.

Gate-level

The project uses gate-level-style logic in the block break and unbreak system. In `top_vga.v`, the design creates signals such as `all_broken`, `newly_broken`, and `unbreak` using Boolean operators that correspond directly to hardware gates. For example, `all_broken = &broken` acts like a large AND gate that checks whether every block has been broken. The `unbreak` signal is created by OR-ing together multiple reset conditions, including all blocks being broken, the cheat-code reset, and the lives-based reset. This means the block array can be recreated whenever any one of those reset conditions becomes true.

The block modules also use gate-level-style logic to decide when an individual block should break. In `block.v`, the signals `blk_lft`, `blk_rgt`, `blk_up`, and `blk_dn` are generated with reduction OR logic from the occupied pixels around the block. These signals are then OR-ed together so that if the ball touches the block from any side, the block's broken register is set. The `unbreak` input clears the broken register back to zero, causing the block to reappear. This break/unbreak logic is gate-level in style because it is built from simple AND, OR, and NOT operations that directly represent the hardware conditions for destroying or recreating blocks.

Dataflow

The project uses dataflow modeling through continuous assign statements that describe combinational logic directly with Boolean expressions and conditional operators. For example, `ball.v`, `block.v`, and `paddle.v` use assign statements to determine whether the current VGA pixel should draw the ball, a block, or the paddle. Other examples include assigning wall detection, empty-pixel logic, score selection, block reset conditions, FIFO empty and full flags, and VGA RGB output values. These statements continuously drive outputs based on the current input and internal signal values, which is the main purpose of dataflow modeling.

For-generate Usage

The project uses a generate block in `top_vga.v` to create the 24 breakable blocks in the game. Instead of manually writing 24 separate block instances, the design uses nested for loops over two rows and twelve columns. Each generated block module receives a calculated `xloc` and `yloc`, allowing the design to place the blocks evenly on the VGA display. This makes the code shorter, cleaner, and easier to modify because the number or layout of blocks can be changed by editing the loop bounds and position formulas instead of rewriting many module instances.

Parameters and Localparams

The project uses parameter and localparam values to make the design easier to configure and read. Several modules use parameters for reusable module settings, such as the starting position and size of the ball, paddle, and blocks. This allows the same module to be reused with different dimensions or locations. The code also uses localparam values for fixed internal constants such as FSM state encodings, VGA timing values, UART baud-rate settings, color values, and score limits. These named constants make the code more understandable and reduce the chance of mistakes from repeated hard-coded numbers.

Explanation of Testbench Strategy & Screenshots of Simulation

Buttons: The buttons function as movement control for the paddle. The buttons are configured as **move_dir[3:0]**. When a button is being held down, the location of the paddle changes, which is monitored through **platform_xloc[9:0]** and **platform_yloc[9:0]**. The left and right buttons decrease or increase **platform_xloc**. Likewise, the up and down buttons increase or decrease **platform_yloc**.

> move_dir[3:0]	0												
		0	1		2	4		8	0				
> platfor...oc[9:0]	140	140		13f				140					
> platfor...oc[9:0]	190	190				191		190					

LEDs: The player's life counter is adjust through force **dut.u_life_fsm.state = 2'dn**, with **n** being the number of lives. Displayed in **led[2:0]**, the value decreases from 7 to 3, 1, and 0, which simulates the player losing a life and the LEDs displaying the lives left in binary.

> led[2:0]	0	7	3	1	0
------------	---	---	---	---	---

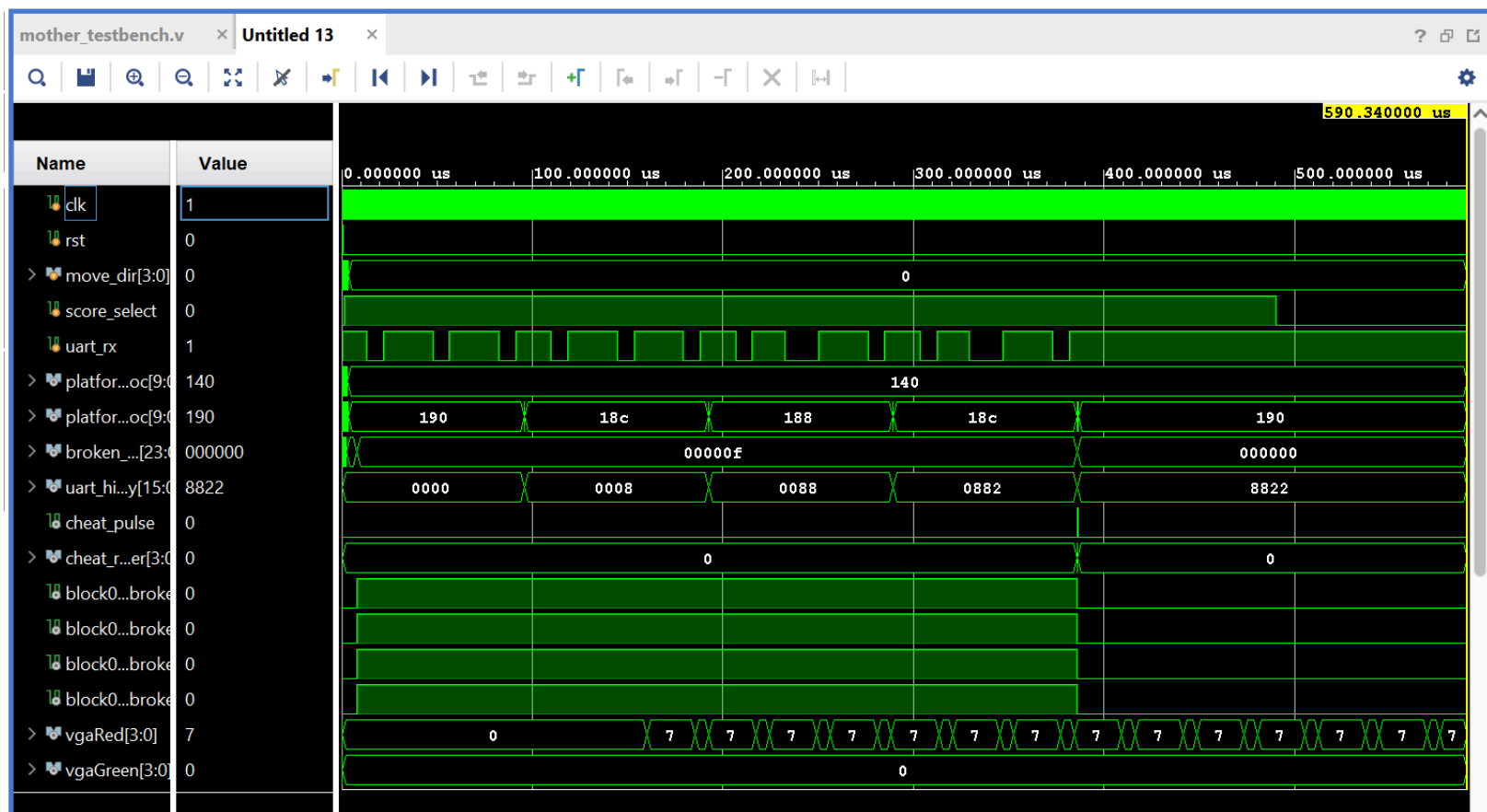
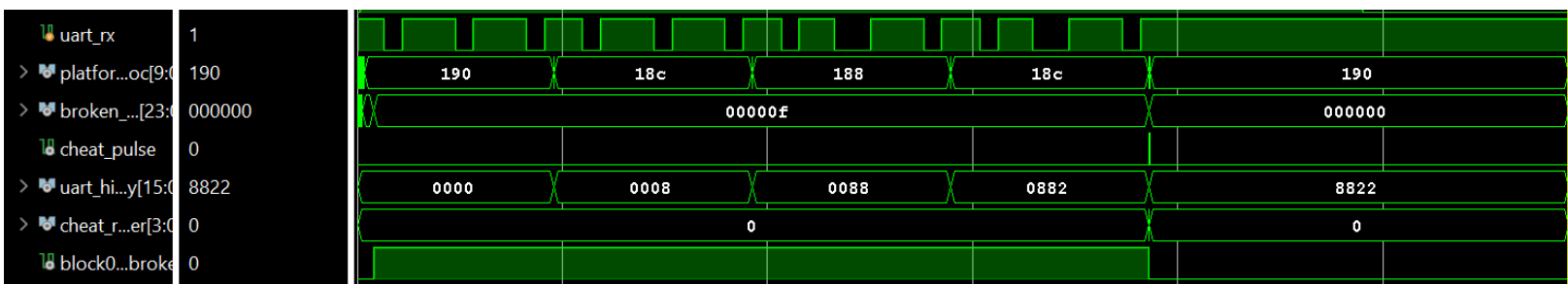
7-Segment Displays & Switch: The display will show two different things: the score of the current game and the total score across all games. When **score_select** is set to 1, the number displayed by the 7-segment displays show 2, which is the score of the current game. When the **score_select** is set to 0, the number on the display is instead 40, which is the total score across all games.

🔍 score_select	0																	
📄 seg[6:0]	40	02																40
🔍 [6]	1																	
🔍 [5]	0																	
🔍 [4]	0																	
🔍 [3]	0																	
🔍 [2]	0																	
🔍 [1]	0																	
🔍 [0]	0																	

VGA: The VGA output is active because the RGB signals are changing instead of staying at a constant blank value. The repeated **hsync** pulses show that the horizontal scan timing is running correctly. Since **vgaRed**, **vgaBlue**, and **hsync** update together, this shows that the VGA controller is scanning across the screen and outputting pixel colors for the game display.

> vgaRed[3:0]	7	0	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7
> vgaGreen[3:0]	0	0																	
> vgaBlue[3:0]	0	f	f	f	f	f	0	0	0	0	0	0	0	0	0	0	0	0	0
7 hsync	1																		
7 vsync	1																		

UART & FIFO: UART is for several purposes. Its first purpose is to serve as a secondary controller along with the buttons. WASD is used for up, left, down, and right movement. To test this, we sent `send_uart_byte("w")` followed by `send_uart_byte("s")` through the UART and observed as `platform_yloc` decreased and increased in value. The second purpose is to implement a cheat code with FIFO, storing the pattern WWSS. If ever this cheat code was typed, all destroyed blocks will respawn, as indicated by `broken_blocks[23:0]` resetting back to 0 after the cheat code was sent.



Brief Conclusion

Overall, this project was both very fun and challenging. We got to implement many of the concepts we learned throughout the course all at once, and it was really cool to see everything come together on actual hardware. From using VGA to display the game's graphics on a monitor, to controlling the paddle with push buttons and a keyboard through UART, to tracking lives with the LEDs and score with the 7-segment displays. Every peripheral we learned about had a real purpose in the game. We also got creative with features like a cheat code that resets the blocks when you type "WWSS" on the keyboard, and a switch that toggles between your current round score and your accumulated score across all rounds. Seeing all of these pieces work together in a game we built from scratch made the project feel rewarding, and we came out of it with a much deeper understanding of Verilog and FPGA.